Generalización de Código de Automatización en Desarrollo de Software con IA

Tema 6. Optimización de algoritmos con inteligencia artificial en desarrollo

Índice

Esquema

Ideas clave

6.1. Introducción y objetivos

6.2. Principios de optimización de algoritmos

6.3. Algoritmos genéticos y optimización por enjambre

6.4. Aplicaciones en optimización de algoritmos de software

6.5. Ajuste de hiperparámetros y mejora continua

6.6. Evaluación del rendimiento con optimización de IA

6.7. Referencias bibliográficas

A fondo

Algoritmos para problemas de optimización

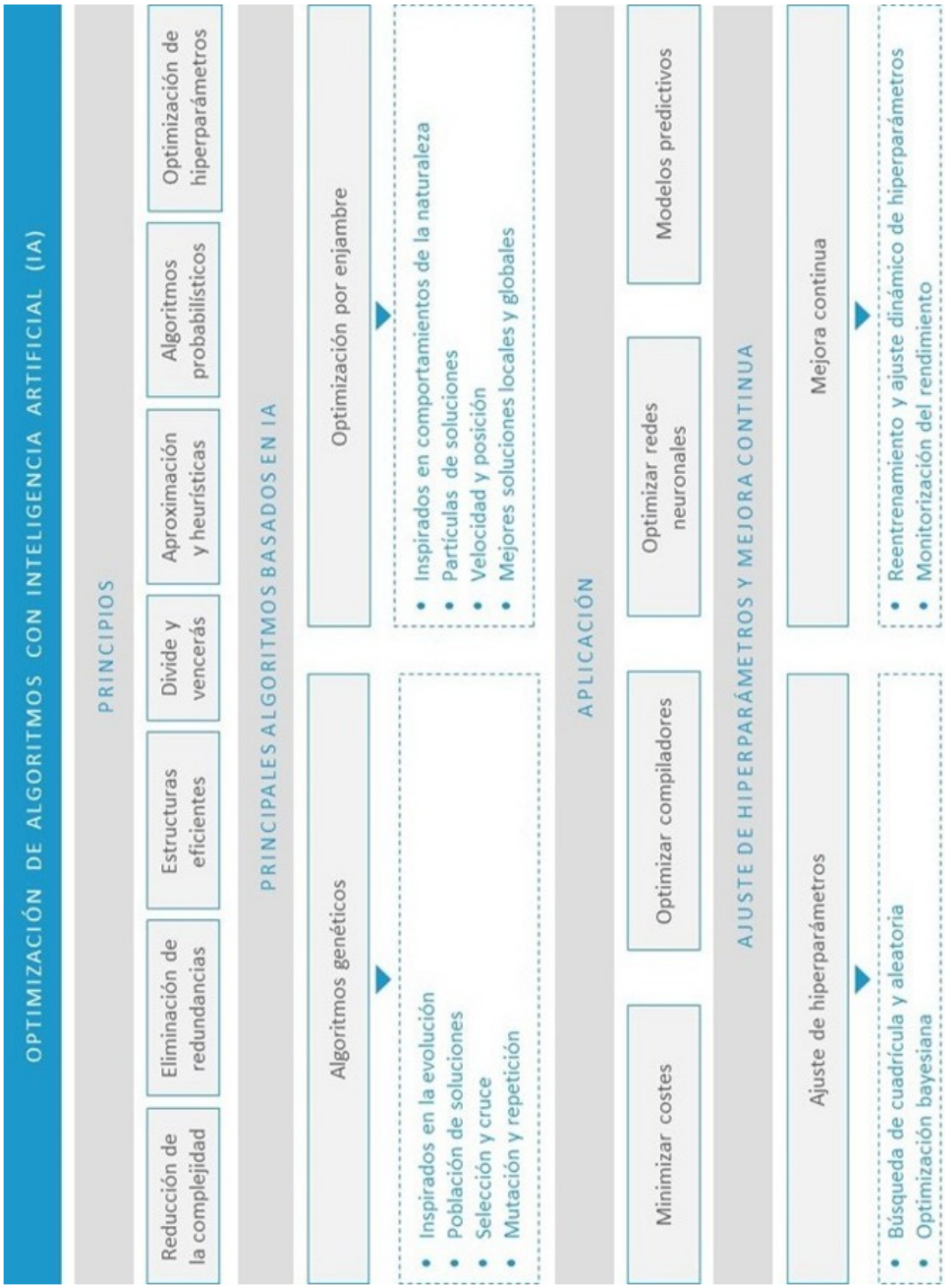
Algoritmos genéticos con ejemplos

Optimización de código con IA

Ajuste de hiperparámetros automática con Amazon SageMaker

Test

Esquema



© Universidad Internacional de La Rioja (UNIR)

6.1. Introducción y objetivos

La **optimización de algoritmos con inteligencia artificial** (IA) es un área clave en el desarrollo de *software* moderno. De esta manera, consiste en **mejorar el rendimiento y la eficiencia de los programas** a través de técnicas de IA para ajustarlos, hacerlos más rápidos y adaptativos. La optimización de algoritmos abarca l a **minimización del tiempo de ejecución,** la reducción del uso de recursos y la mejora continua mediante el ajuste automático de parámetros y la actualización constante.

Los objetivos de este tema son:

- Comprender los principios fundamentales de la optimización de algoritmos y cómo la IA mejora este proceso, lo que permite soluciones eficientes.
- Explorar técnicas avanzadas de optimización, como los algoritmos genéticos, el enjambre de partículas y las técnicas de aprendizaje por refuerzo.
- Aplicar estrategias para el ajuste de hiperparámetros en modelos de *machine learning,* para garantizar un rendimiento óptimo y adaptativo.
- Evaluar el impacto de la optimización en el rendimiento de *software* y aprender a medir su efectividad mediante métricas clave.
- Implementar casos prácticos de optimización de algoritmos en aplicaciones como la planificación de tareas, la optimización de redes neuronales y simulaciones.

© Universidad Internacional de La Rioja (UNIR)

6.2. Principios de optimización de algoritmos

La **optimización de algoritmos** es un proceso esencial en el desarrollo de *software,* ya que se busca mejorar el rendimiento y la eficiencia de los programas. Con la **integración de IA,** este proceso evolucionó, ya que permitió el uso de **técnicas avanzadas** que mejoran aún más los algoritmos.

La optimización de algoritmos se centra en mejorar la eficiencia y el rendimiento de los programas para **reducir el tiempo de ejecución,** el **uso de memoria,** y otros recursos.

Los principios clave de la optimización se basan en técnicas que buscan minimizar el costo computacional y mejorar la escalabilidad de los algoritmos.

Este proceso se basa en unos principios que, aplicados con las técnicas adecuadas de IA, permiten una **optimización eficaz de algoritmos** en desarrollo de *software,* lo que mejora tanto el rendimiento como la eficiencia. A continuación, se detallan los **principios clave** para aplicar la IA en la optimización de algoritmos, sus técnicas y casos prácticos.

Reducción de la complejidad temporal y espacial

La complejidad temporal se refiere al **tiempo que tarda** un algoritmo en ejecutarse, mientras que la complejidad espacial se refiere a la **cantidad de memoria utilizada.** La optimización busca **reducir estas complejidades.**

Por ejemplo, para ordenar una lista, el algoritmo de búsqueda binaria tiene una complejidad de $O(\log n)$, lo cual es más eficiente que una búsqueda lineal de $O(n)$. Si se necesita almacenar una gran cantidad de datos, se pueden utilizar estructuras de datos, como árboles balanceados, para minimizar el uso de memoria.

© Universidad Internacional de La Rioja (UNIR)

Eliminación de redundancias

Los algoritmos pueden contener **operaciones redundantes** o cálculos repetidos que pueden ser eliminados o simplificados.

Por ejemplo, en el cálculo del factorial de un número, la técnica de almacenamiento en caché o memorización se puede aplicar para guardar los resultados de subproblemas previamente calculados, para evitar recalculaciones.

Uso de estructuras de datos eficientes

Seleccionar la **estructura de datos adecuada** para una tarea puede mejorar significativamente la eficiencia del algoritmo.

Por ejemplo, para operaciones de búsqueda y acceso rápido, una tabla *hash* puede ser más eficiente que una lista o arreglo. En el caso de tareas de prioridad, un montículo *(heap)* es más adecuado.

Divide y vencerás

Este enfoque implica dividir el problema en subproblemas más pequeños, para resolverlos de forma recursiva y combinar sus soluciones.

Por ejemplo, el algoritmo de ordenamiento rápido *(quicksort)* divide una lista en dos sublistas y ordena cada sublista de manera recursiva, de manera que logra una complejidad promedio de $O(n \log n)$.

© Universidad Internacional de La Rioja (UNIR)

Aproximación y heurísticas

E n **problemas complejos,** donde encontrar la solución óptima es costoso, se pueden utilizar algoritmos aproximados o heurísticas para encontrar una **solución cercana al óptimo** en un tiempo razonable.

Por ejemplo, para el problema del viajante de comercio, se pueden usar algoritmos heurísticos, como algoritmos genéticos (GA) o recocido simulado, para encontrar una solución que minimice la distancia recorrida sin evaluar todas las rutas posibles.

Uso de algoritmos probabilísticos

Los algoritmos probabilísticos o aleatorizados utilizan **decisiones basadas en la probabilidad** para reducir el tiempo de ejecución o mejorar la eficiencia.

Por ejemplo, el algoritmo de Monte Carlo se utiliza en simulaciones para calcular integrales complejas o resolver problemas de optimización, por lo que proporciona una solución rápida, aunque no siempre exacta.

Optimización de hiperparámetros en modelos de *machine learning*

E n *machine learning,* la optimización de hiperparámetros implica encontrar la **configuración adecuada** de los parámetros que no se aprenden durante el entrenamiento, como la **tasa de aprendizaje** o el **número de capas de una red neuronal.**

Por ejemplo, el uso de técnicas como la búsqueda en cuadrícula, la búsqueda aleatoria y la optimización bayesiana permite encontrar la mejor combinación de hiperparámetros para mejorar la precisión del modelo.

© Universidad Internacional de La Rioja (UNIR)

Técnicas de optimización de algoritmos

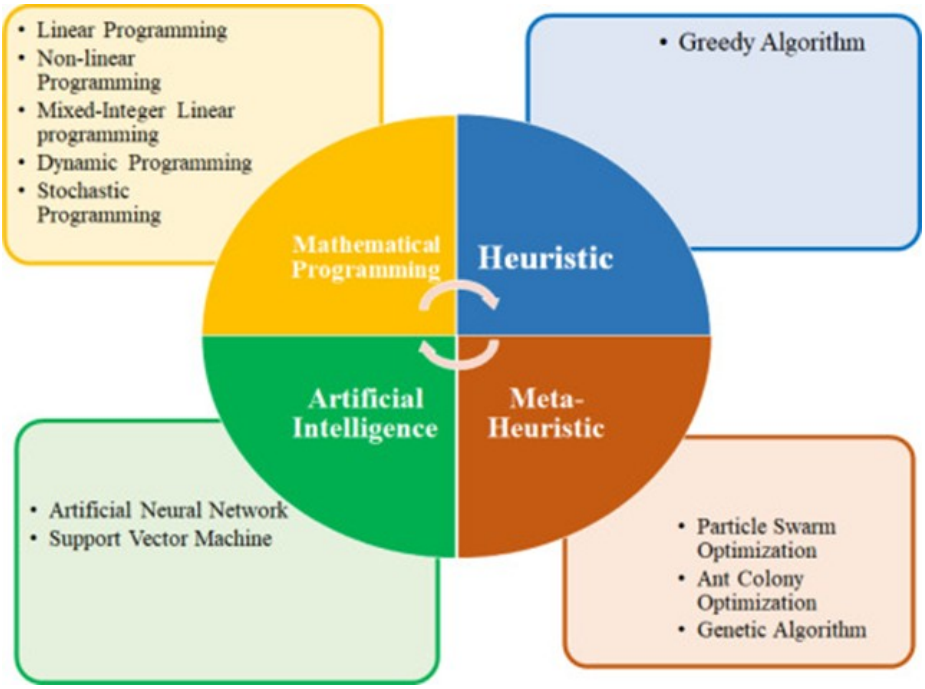


Figura 1. Técnicas aplicadas en optimización de algoritmos. Fuente: Nutakki y Mandava, 2023.

Las técnicas de optimización de algoritmos aplicadas al desarrollo *software* permiten mejorar la eficiencia operativa y **reducir costos en entornos dinámicos.** Al aprovechar técnicas matemáticas, heurísticas y de IA, se pueden **automatizar y optimizar tareas críticas** en el ciclo de vida del *software,* de manera que se logra obtener un **flujo de trabajo más ágil y adaptable.**

Estas técnicas permiten que las organizaciones aprovechen mejor los recursos y reduzcan el tiempo de desarrollo y de despliegue de *software.*

© Universidad Internacional de La Rioja (UNIR)

Optimización matemática

La optimización matemática **involucra métodos** como la programación lineal, la no lineal, la programación cuadrática y la programación dinámica que se utilizan para **resolver problemas complejos,** así como para minimizar costos o maximizar recursos. Su aplicación en DevOps se da de la siguiente manera:

- Planificación de recursos. La programación lineal puede aplicarse para optimizar la asignación de recursos en entornos de nube, lo que determina la cantidad de recursos que deben desplegarse para cumplir con los requerimientos de la aplicación sin incurrir en costos innecesarios.

- Minimización de tiempo de compilación y despliegue. Las técnicas de programación cuadrática y dinámica pueden ayudar a reducir el tiempo total necesario para las compilaciones y despliegues, por lo que optimizan la secuencia en la que se ejecutan las tareas.

Técnicas heurísticas y metaheurísticas

Las técnicas heurísticas y metaheurísticas, como los GA, la optimización por enjambre de partículas (PSO), la optimización por colonia de hormigas (ACO) y el algoritmo de búsqueda armónica, buscan **soluciones aproximadas** para problemas complejos mediante la **exploración del espacio de soluciones** de manera eficiente. Su aplicación en DevOps es la siguiente:

- Automatización de pruebas. Los GA se pueden utilizar para optimizar la generación de casos de prueba a través de la selección automática de los casos más relevantes para reducir el tiempo de ejecución de las pruebas.

© Universidad Internacional de La Rioja (UNIR)

- Optimización de canalizaciones de CI (integración continua) y CD (distribución continua). La PSO puede aplicarse para ajustar automáticamente los parámetros de las canalizaciones de CI y CD. De esta manera, encuentra la configuración óptima que minimizará los tiempos de ejecución y maximizará la calidad del despliegue.

- Asignación de tareas en microservicios. La ACO puede ayudar a asignar tareas a diferentes microservicios de manera óptima, lo que minimiza la latencia y el consumo de recursos.

Optimización basada en aprendizaje automático

El aprendizaje automático *(machine learning)* utiliza técnicas como redes neuronales artificiales (ANN) y aprendizaje por refuerzo para adaptar dinámicamente las estrategias de optimización en función de los datos históricos y en tiempo real.

El aprendizaje por refuerzo emplea premios y penalizaciones para ayudar a entrenar un modelo de IA, de forma que se recompensa cuando se introducen mejoras y se penaliza cuando los cambios son para peor.

Además, la IA puede mejorar la eficiencia y la calidad en el ciclo de vida de DevOps a través de la **aplicación de técnicas avanzadas** para la optimización de algoritmos que abordan tareas específicas. Su aplicación en DevOps es la siguiente:

- Predicción de demandas de recursos. Los modelos de redes neuronales se pueden entrenar con datos históricos de uso para predecir las demandas futuras de recursos, lo que permite ajustar dinámicamente la capacidad de la infraestructura según sea necesario.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- Optimización de flujos de trabajo. El aprendizaje por refuerzo se puede utilizar para optimizar flujos de trabajo en CI y CD, de manera que aprende continuamente cuál es la mejor secuencia de pasos y los parámetros para lograr despliegues rápidos y sin errores.

- Monitoreo inteligente. Mediante técnicas de aprendizaje automático, se puede implementar el análisis predictivo para detectar problemas de rendimiento o fallas potenciales antes de que ocurran, para activar automáticamente acciones correctivas.

- Automatización del diagnóstico de errores. El uso de algoritmos de aprendizaje profundo puede ayudar a identificar y a categorizar errores en el código o en el entorno de ejecución, por lo que sugiere soluciones automáticamente.

- Optimización del tiempo de respuesta de aplicaciones. Mediante la IA predictiva, se puede ajustar el escalado automático de aplicaciones para garantizar un rendimiento constante, lo que optimiza el uso de recursos en función de la carga.

- Orquestación inteligente de despliegues. La IA puede optimizar la orquestación de despliegues, de manera que gestiona cuándo y cómo se realizan las actualizaciones de *software* para minimizar el tiempo de inactividad.

© Universidad Internacional de La Rioja (UNIR)

Ejemplos de implementación en DevOps

- Uso de GA para optimizar configuraciones de CI y CD. En un entorno de CI y CD, la configuración de los parámetros, como el tiempo de espera de las pruebas o el número de trabajos concurrentes, puede afectar el tiempo total de despliegue. A través de un algoritmo genético, se pueden explorar diferentes configuraciones para encontrar la combinación óptima que minimice el tiempo sin comprometer la calidad.

- PSO para escalado automático. En un entorno de nube, la PSO se puede utilizar para determinar el número óptimo de instancias de servidor que deben estar activas en un momento dado, para lo que hay que tener en cuenta la carga actual y las predicciones futuras, lo que ayuda a reducir costos operativos.

- Aprendizaje por refuerzo para la implementación de flujos de trabajo adaptativos. Mediante un aprendizaje por refuerzo, se puede entrenar un sistema para optimizar la secuencia de tareas en un flujo de trabajo de DevOps, para adaptar las etapas en función del resultado de despliegues anteriores, mejorar continuamente la eficiencia y reducir los errores.

© Universidad Internacional de La Rioja (UNIR)

6.3. Algoritmos genéticos y optimización por enjambre

La optimización de algoritmos es un campo clave dentro del desarrollo de *software,* especialmente cuando se buscan **soluciones óptimas** en espacios de búsqueda complejos o problemas multidimensionales. Dos técnicas destacadas en este ámbito son los **GA** y la **PSO.** Ambas se inspiran en procesos naturales y se utilizan para **resolver problemas de optimización difíciles,** donde las soluciones exactas pueden ser imposibles o demasiado costosas de calcular.

Estos dos enfoques de IA para la optimización tienen amplias aplicaciones en el desarrollo de *software* y otras áreas, lo que permite la mejora continua de algoritmos en sistemas complejos.

A continuación, se detallan sus principios, funcionamiento y ejemplos de aplicación.

© Universidad Internacional de La Rioja (UNIR)

GA

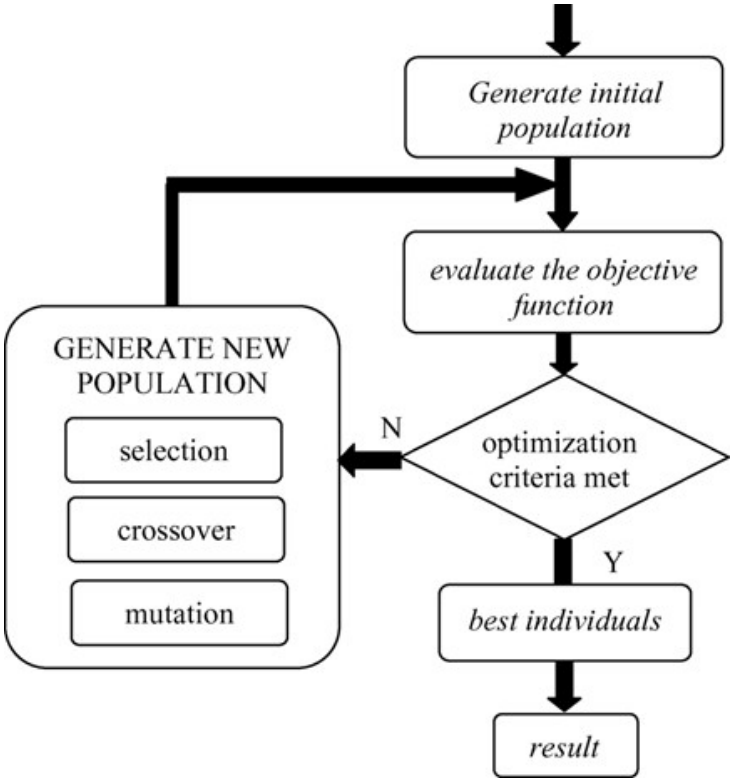


Figura 2. Diagrama de funcionamiento de un GA. Fuente: Ozdemir et al, 2005.

Los GA son una técnica de optimización inspirada en la **teoría de la evolución de Darwin.** En este enfoque, se simula el proceso evolutivo mediante la creación de una población de posibles soluciones, las cuales se **seleccionan, combinan y mutan** a lo largo de múltiples generaciones para mejorar su rendimiento en función de un criterio de aptitud, lo que les permite **explorar un espacio de soluciones muy amplio.**

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Principios clave de los GA

- Población inicial. Se genera un conjunto aleatorio de soluciones iniciales llamadas individuos o cromosomas.
- Selección. Los individuos más aptos (los que mejor resuelven el problema) se seleccionan para reproducirse, para generar nuevas soluciones.
- *Crossover* (cruce). Las soluciones seleccionadas se combinan para producir descendientes, que heredan las características de ambas soluciones parentales.
- Mutación. A ciertos individuos, se les aplican pequeñas modificaciones aleatorias para explorar nuevas soluciones que no estaban en la población original.
- Repetición. Este proceso se repite durante varias generaciones, con la esperanza de que las nuevas generaciones mejoren la calidad de las soluciones con respecto a las generaciones anteriores.

Funcionamiento general de un GA

- Generación de población inicial. Se define una población inicial de soluciones potenciales.
- Evaluación de aptitud. Cada solución es evaluada a través de una función de aptitud, que mide qué tan cerca está de la solución óptima.
- Selección de padres. Se seleccionan las mejores soluciones de la población con base en su aptitud.
- *Crossover* y mutación. Se realizan cruces y mutaciones para generar nuevas soluciones (descendientes).
- Reemplazo. Los descendientes sustituyen a la población inicial y el ciclo se repite hasta que se alcanza un criterio de parada (por ejemplo, un número determinado de generaciones o una solución suficientemente buena).

© Universidad Internacional de La Rioja (UNIR)

Ejemplos de aplicación de GA

- Optimización de rutas. Los GA se utilizan para resolver el problema del viajante de comercio, en el cual se busca la ruta más corta para visitar un conjunto de ciudades.
- Diseño de redes. En la optimización de redes de telecomunicaciones, los GA ayudan a encontrar la mejor configuración de topología y asignación de recursos.
- Planificación de tareas. Los GA pueden optimizar la asignación de tareas en un entorno de servidores, de manera que distribuyen eficientemente las cargas de trabajo.

PSO

La PSO es una técnica de optimización inspirada en el comportam**iento colectivo de animales,** como bandadas de pájaros o bancos de peces. En PSO, se crea un enjambre de partículas que exploran el espacio de soluciones, de manera que **ajustan sus posiciones** en función de su experiencia individual y la experiencia compartida por el grupo.

La PSO puede compararse con el comportamiento de las neuronas en el cerebro, ya que ajustan su actividad en función de las señales de otras neuronas y de su experiencia previa.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Principios clave de la PSO

- Partículas. Cada partícula representa una posible solución al problema.
- Velocidad y posición. Las partículas se mueven a través del espacio de búsqueda, por lo que ajustan su velocidad y posición según su experiencia pasada y la mejor posición encontrada por el enjambre.
- Mejores soluciones. Cada partícula mantiene un registro de su mejor solución local y el enjambre entero recuerda la mejor solución global encontrada hasta el momento.

Funcionamiento general de la PSO

- Inicialización. Se define un conjunto inicial de partículas, cada una con una posición y velocidad iniciales en el espacio de búsqueda.
- Evaluación de partículas. Cada partícula evalúa la calidad de su posición actual a través de una función objetivo.
- Actualización de velocidades y posiciones. Las partículas ajustan su velocidad en función de su propia experiencia y de la mejor posición encontrada por el enjambre.
- Búsqueda de la mejor solución. El proceso se repite hasta que el enjambre converja hacia una solución óptima o se alcance un criterio de parada.

Ejemplos de aplicación de la PSO

- Optimización de hiperparámetros en IA. La PSO es útil para encontrar los mejores valores de hiperparámetros en redes neuronales profundas o modelos de *machine learning,* por lo que ajusta eficientemente las variables de entrada.
- Optimización de sistemas energéticos. La PSO se puede usar para optimizar la distribución de energía en redes eléctricas inteligentes, por lo que se ajusta la carga y la producción en tiempo real.

© Universidad Internacional de La Rioja (UNIR)

- Optimización de diseños mecánicos. En el diseño asistido por computadora (CAD), la PSO ayuda a optimizar parámetros geométricos y estructurales de los componentes.

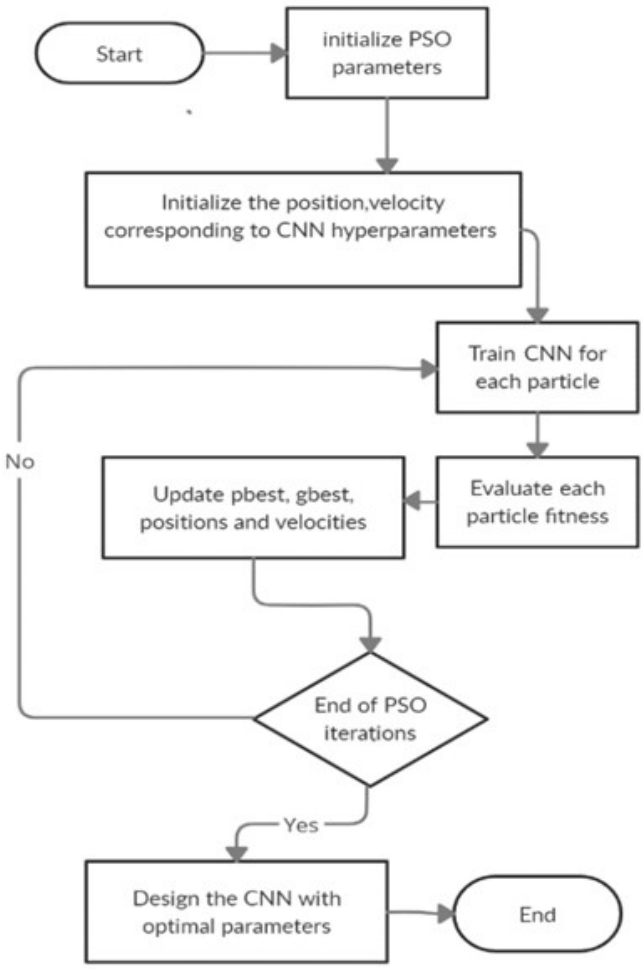


Figura 3. La PSO aplicada a la optimización de hiperparámetros en CNN (red neuronal convolucional).

Fuente: Fouad et al, 2021.

© Universidad Internacional de La Rioja (UNIR)

Comparación entre GA y PSO

Característica	GA	PSO
Inspiración	Evolución biológica (selección natural)	Comportamiento social de enjambres
Estrategia de optimización	Exploración mediante cruces y mutaciones	Movimiento continuo en el espacio de soluciones
Evolución	Evolución por generaciones y reemplazo de individuos	Evolución continua de partículas
Aplicabilidad	Problemas de optimización combinatoria compleja	Problemas multidimensionales y continuos
Convergencia	Más lenta, pero es capaz de escapar de mínimos locales	Rápida, pero con mayor riesgo de converger en mínimos locales

Tabla 1. Comparación entre GA y PSO. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

6.4. Aplicaciones en optimización de algoritmos de software

La optimización de algoritmos de *software* es esencial para **mejorar el rendimiento, la eficiencia y la escalabilidad** de las aplicaciones. La IA, especialmente a través de técnicas de aprendizaje automático y de algoritmos evolutivos, amplió significativamente las capacidades para **optimizar algoritmos** en áreas como el ajuste de hiperparámetros, la minimización de tiempos de procesamiento y la mejora del uso de recursos.

Estas técnicas les permiten a los desarrolladores crear sistemas más rápidos, eficientes y escalables que se adaptan dinámicamente a las necesidades y las limitaciones del entorno en que se ejecutan.

A continuación, se presentan algunas de las principales aplicaciones de IA en la optimización de algoritmos de *software.*

Minimización de tiempos de ejecución

La IA se utiliza para optimizar el rendimiento de algoritmos, lo que **minimiza los tiempos de procesamiento** en diferentes etapas de ejecución. Una técnica común es la **identificación de cuellos de botella** mediante el análisis de patrones de ejecución y la aplicación de algoritmos que **ajusten dinámicamente** las partes del código que causan mayores retrasos.

Por ejemplo, en bases de datos distribuidas, la IA puede **analizar consultas SQL** *(structured query language)* complejas y **optimizar el orden de ejecución** de los comandos, lo que mejora drásticamente el tiempo de respuesta de las consultas.

© Universidad Internacional de La Rioja (UNIR)

Optimización de compiladores

Los **compiladores inteligentes** pueden usar IA para generar automáticamente código optimizado, por lo que ajustan cómo se **traduce** el código fuente en código máquina. Esto puede incluir la **reorganización de instrucciones,** reducción de ciclos y minimización del uso de memoria.

Por ejemplo, en el desarrollo de sistemas embebidos, los compiladores que utilizan técnicas de *machine learning* pueden generar código más eficiente para *hardware* con recursos limitados, lo que resulta en mejoras sustanciales en el rendimiento del *software.*

El compilador de código de Meta, LLM *(large language model)*, es una herramienta que utiliza técnicas avanzadas de IA para **traducir instrucciones en lenguaje natural** o código de alto nivel a **código ejecutable** en un lenguaje de programación específico. A diferencia de los compiladores tradicionales, que se limitan a traducir código fuente a código máquina o *bytecode,* el LLM aprovecha el procesamiento del lenguaje natural para **generar, optimizar y corregir código.**

© Universidad Internacional de La Rioja (UNIR)

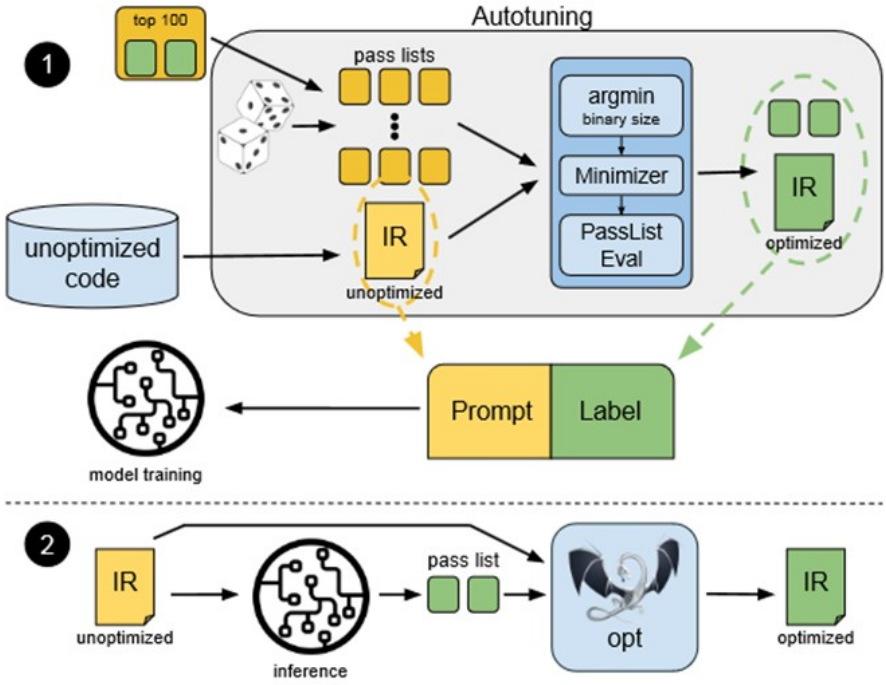


Figura 4. Funcionamiento del compilador LLM. Fuente: Cummins et al., 2024.

Optimización en redes neuronales

La optimización de redes neuronales puede incluir la *pruning* (poda de redes) y la cuantización.

Estas técnicas permiten **reducir la complejidad** del modelo sin comprometer significativamente su precisión, por lo que **mejoran el rendimiento** en términos de velocidad y uso de memoria.

Por ejemplo, en aplicaciones de visión por computadora, las redes convolucionales profundas (CNN) a menudo requieren mucha capacidad de cómputo. Con técnicas de poda, se puede reducir la cantidad de parámetros, lo que permite ejecutar los modelos en dispositivos con recursos limitados, como teléfonos móviles.

© Universidad Internacional de La Rioja (UNIR)

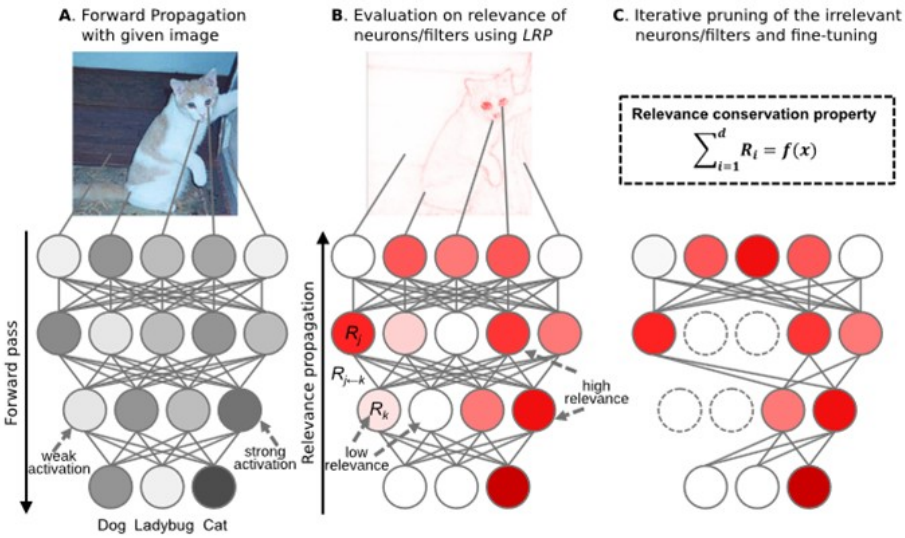


Figura 5. Optimización de CNN mediante poda. Fuente: Yeom et al., 2021.

La cuantización permite **ajustar la precisión de los valores** de los parámetros mediante la conversión de valores continuos a discretos, generalmente se cambia el tipo de datos de int32 a int8. Por otro lado, con la poda, se **eliminan conexiones, neuronas o filtros** en una red neuronal para reducir su tamaño sin afectar significativamente su rendimiento.

Optimización en simulaciones y modelos predictivos

Las simulaciones complejas, como las usadas en modelos financieros o predicciones climáticas, pueden ser **optimizadas mediante IA** para mejorar la precisión y reducir los tiempos de cálculo.

Por ejemplo, en simulaciones climáticas, la IA puede reducir significativamente los tiempos de simulación, de manera que optimiza las ecuaciones diferenciales que describen los fenómenos meteorológicos, sin sacrificar la precisión de los resultados.

© Universidad Internacional de La Rioja (UNIR)

Caso práctico: optimización de consultas SQL complejas a través de la PSO

Contexto del caso práctico

- Objetivo. Utilizar un algoritmo PSO para optimizar la cláusula `WHERE` de una consulta SQL. El objetivo es reordenar los predicados de la cláusula `WHERE` para reducir el tiempo de respuesta de la consulta.

- Escenario. Se tiene una base de datos con una tabla llamada ventas, que almacena millones de registros. La consulta SQL que se optimizará contiene múltiples predicados en la cláusula `WHERE` , pero su tiempo de ejecución es lento. El objetivo es encontrar el orden óptimo de ejecución de los predicados para minimizar el tiempo total de respuesta.

Modelo de consulta original:

```
SELECT *

FROM ventas

WHERE condicion1 AND condicion2 AND condicion3 AND condicion4;
```

El **orden de evaluación** de los predicados en la cláusula `WHERE` afecta el tiempo de ejecución. Si se evalúan primero las condiciones que filtran más registros, se puede **reducir la cantidad de registros** que pasan a las condiciones siguientes, lo que mejora la eficiencia.

© Universidad Internacional de La Rioja (UNIR)

Requisitos:

- Utilizar la PSO para optimizar el orden de evaluación de los predicados.
- Evaluar el tiempo de ejecución de la consulta para cada posible orden de evaluación.
- Mostrar la mejor secuencia de predicados junto con el tiempo de respuesta optimizado.

Instalación de bibliotecas

Instalamos las bibliotecas necesarias para la simulación: `$> pip install pyswarm sqlalchemy`

Explicación de las bibliotecas:

- `pyswarm` es la implementación del algoritmo de PSO en Python.
- `sqlalchemy` se utiliza para conectar y ejecutar consultas SQL en la base de datos.

Configuración de la base de datos

Para la simulación, creamos una base de datos SQLite con una tabla llamada ventas.

```
import sqlite3

import pandas as pd

import random

# Crear la conexión con la base de datos SQLite

conn = sqlite3.connect('ventas.db')



# Crear la tabla de ventas
```


© Universidad Internacional de La Rioja (UNIR)

```
conn.execute('''
CREATE TABLE IF NOT EXISTS ventas (
    id INTEGER PRIMARY KEY,
    producto TEXT,
    categoria TEXT,
    precio REAL,
    cantidad INTEGER,
    fecha_venta TEXT
)
''')
# Insertar registros de ejemplo
productos = ['Producto_A', 'Producto_B', 'Producto_C', 'Producto_D']
categorias = ['Categoria_1', 'Categoria_2', 'Categoria_3']
fechas = pd.date_range(start='2022-01-01', end='2022-12-31', freq='D')

for _ in range(100000): # Insertar 100,000 registros
    producto = random.choice(productos)
    categoria = random.choice(categorias)
    precio = round(random.uniform(10, 1000), 2)
    cantidad = random.randint(1, 100)
```


© Universidad Internacional de La Rioja (UNIR)

```
    fecha_venta = random.choice(fechas).strftime('%Y-%m-%d')

    conn.execute('INSERT INTO ventas (producto, categoria, precio,
cantidad, fecha_venta) VALUES (?, ?, ?, ?, ?)',

                 (producto, categoria, precio, cantidad, fecha_venta))

conn.commit()

conn.close()
```

Definición de la PSO para la optimización

Parámetros de la PSO:

- Partícula (posición). Cada partícula es un posible orden de los predicados de la cláusula WHERE .
- Función de fitness . La función de aptitud mide el tiempo de respuesta de la consulta SQL con un orden específico de los predicados.
- Objetivo. Minimizar el tiempo de respuesta de la consulta.

Predicados para la cláusula WHERE :

```
WHERE producto = 'Producto_A'

  AND categoria = 'Categoria_1'

  AND precio > 50

  AND cantidad < 20
```

Implementación del código completo

```
import sqlite3

import time
```


© Universidad Internacional de La Rioja (UNIR)

```
import random

from pyswarm import pso

# Conectar con la base de datos SQLite

conn = sqlite3.connect('ventas.db')



# Lista de predicados de la cláusula WHERE

predicados = [

    "producto = 'Producto_A'",

    "categoria = 'Categoria_1'",

    "precio > 50",

    "cantidad < 20"

]

# Función que evalúa el tiempo de ejecución de una consulta SQL con un
orden de predicados específico

def evaluar_consulta_sql(orden):

    # Reordenar los predicados según el orden proporcionado por la
partícula

    predicados_ordenados = [predicados[int(i)] for i in orden]

    where_clause = " AND ".join(predicados_ordenados)

    consulta_sql = f"SELECT * FROM ventas WHERE {where_clause}"
```


© Universidad Internacional de La Rioja (UNIR)

```
    # Medir el tiempo de ejecución de la consulta
    inicio = time.time()
    conn.execute(consulta_sql).fetchall()
    fin = time.time()

    tiempo_respuesta = fin - inicio
    return tiempo_respuesta
# Función de fitness para la PSO (minimizar el tiempo de respuesta de la
consulta)
def funcion_fitness(orden):
    tiempo = evaluar_consulta_sql(orden)
    return tiempo
# Definir el espacio de búsqueda (rango de índices de predicados)
dimensiones = len(predicados)
limites_inferiores = [0] * dimensiones
limites_superiores = [len(predicados) - 1] * dimensiones
# Ejecutar la PSO para encontrar el mejor orden de predicados
mejor_orden, mejor_tiempo = pso(
    funcion_fitness,
    lb=limites_inferiores,
    ub=limites_superiores,
```


© Universidad Internacional de La Rioja (UNIR)

```
    swarmsize=10,
    maxiter=20
)
# Mostrar el mejor orden de los predicados y el tiempo de respuesta
mejor_orden = [int(i) for i in mejor_orden]
predicados_ordenados = [predicados[i] for i in mejor_orden]
print(f"Mejor orden de predicados: {predicados_ordenados}")
print(f"Tiempo de respuesta optimizado: {mejor_tiempo:.4f} segundos")
# Cerrar la conexión
conn.close()
```

Explicación del código

- La función `evaluar_consulta_sql(orden)` toma un orden de predicados y ejecuta la consulta SQL, así como calcula el tiempo de respuesta.
- La función `funcion_fitness(orden)` devuelve el tiempo de respuesta, que es el objetivo que hay que minimizar.
- Optimización por PSO:
  - Cada partícula representa un orden posible de los predicados de la cláusula `WHERE`.
  - El objetivo es minimizar el tiempo de respuesta de la consulta.

© Universidad Internacional de La Rioja (UNIR)

Resultado posible

Mejor orden de predicados:

```
["categoria = 'Categoria_1'", "precio > 50", "cantidad < 20", "producto =
'Producto_A'"]
```

```
Tiempo de respuesta optimizado: 0.3524 segundos
```

Conclusiones

- Optimización de la cláusula `WHERE` . El orden de los predicados de la cláusula `WHERE` afecta directamente al tiempo de respuesta de la consulta SQL.
- Efectividad del PSO:
  - La PSO encuentra la secuencia de predicados que mejor reduce el tiempo de consulta.
  - En este caso, el tiempo de consulta se redujo de 1,2 segundos (orden predeterminado) a 0,35 segundos (orden optimizado).
- Aplicaciones:
  - Esta técnica se puede aplicar para optimizar las consultas de bases de datos reales con millones de registros.
  - Los beneficios son claros en términos de eficiencia y de velocidad de respuesta.

© Universidad Internacional de La Rioja (UNIR)

6.5. Ajuste de hiperparámetros y mejora continua

E l **ajuste de hiperparámetros** es una parte fundamental en el proceso de entrenamiento de modelos de aprendizaje automático y de redes neuronales. Los hiperparámetros son los **parámetros externos al modelo** (como la tasa de aprendizaje, el tamaño de los lotes, el número de capas ocultas en una red neuronal, etc.) que deben **configurarse** antes del proceso de entrenamiento.

El objetivo del ajuste de hiperparámetros es encontrar la combinación que maximice el rendimiento del modelo para lograr un equilibrio entre precisión, velocidad y consumo de recursos.

Además del ajuste de hiperparámetros, la mejora continua implica la **actualización constante** del modelo y de su rendimiento a medida que se introducen nuevos datos o se desarrollan **nuevas técnicas,** lo que asegura que el sistema mantenga su eficacia a largo plazo.

Estos procesos son fundamentales en el **desarrollo y mantenimiento** de modelos d e *machine learning.* Elegir correctamente los hiperparámetros puede marcar la diferencia entre un **modelo eficiente** y uno que no logra su cometido. Además, la mejora continua garantiza que los modelos sigan siendo **precisos y relevantes** a medida que los entornos de datos cambian con el tiempo.

Importancia del ajuste de hiperparámetros

El rendimiento de los modelos de aprendizaje automático depende, en gran medida, de los **hiperparámetros seleccionados.** Si los hiperparámetros no se ajustan correctamente, es posible que el modelo **no converja,** que tenga un sobreajuste o subajuste, o que **no aproveche todo su potencial.**

© Universidad Internacional de La Rioja (UNIR)

Principales hiperparámetros que hay que ajustar

- La tasa de aprendizaje *(learning rate)* controla el tamaño de los pasos que da el modelo durante la actualización de pesos. Un valor demasiado alto puede causar que el modelo no converja, mientras que uno muy bajo hará que el entrenamiento sea extremadamente lento.

- El tamaño del lote *(batch size)* determina cuántos ejemplos se procesan antes de que se actualicen los pesos. Los tamaños pequeños pueden generalizar mejor, pero son más lentos; los tamaños grandes son rápidos, pero pueden sobreajustar.

- El número de épocas *(epochs)* indica cuántas veces el modelo verá el conjunto completo de datos. Si es demasiado alto, puede llevar a sobreajuste; si es bajo, puede que no capture los patrones adecuados.

- La arquitectura de la red incluye el número de capas y de neuronas en una red neuronal, o los parámetros de modelos como los árboles de decisión, las redes de apoyo vectorial (SVM), entre otros.

Por ejemplo, en el entrenamiento de una CNN para la clasificación de imágenes, la combinación óptima de la tasa de aprendizaje y el tamaño del lote pueden marcar la diferencia entre un modelo que tarda días en converger y uno que lo hace de manera eficiente en unas pocas horas.

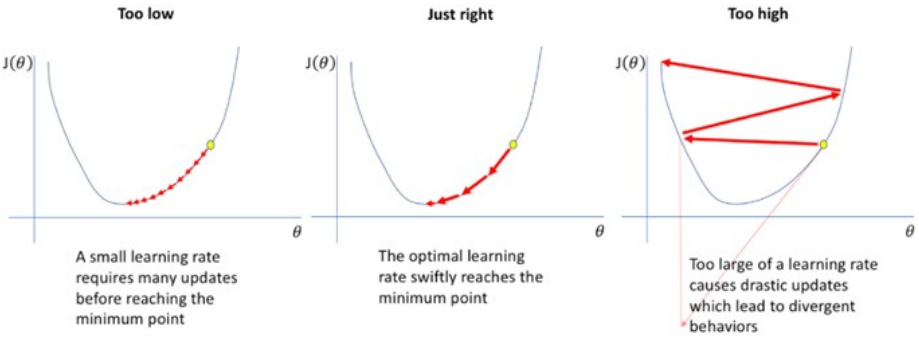


Figura 6. Comparativa de como evaluación un modelo según la tasa de aprendizaje (LR). Fuente: Li, 2024.

© Universidad Internacional de La Rioja (UNIR)

Métodos de ajuste de hiperparámetros

Búsqueda en cuadrícula *(grid search)*

Este método consiste en definir un **rango de valores** para cada hiperparámetro y **evaluar todas las combinaciones posibles.** Aunque es exhaustivo, puede ser costoso en tiempo y recursos cuando hay muchos hiperparámetros.

- La ventaja es que busca de manera sistemática todas las combinaciones posibles.
- La desventaja es que consume mucho tiempo y recursos, especialmente cuando el espacio de hiperparámetros es grande.

Búsqueda aleatoria *(random search)*

En lugar de probar todas las combinaciones, se seleccionan **combinaciones aleatorias** de hiperparámetros. Esto permite **cubrir un espacio amplio** de manera más eficiente.

- La ventaja es que es más rápida que la búsqueda en cuadrícula y suele dar buenos resultados en menos tiempo.
- La desventaja es que la selección es aleatoria, por lo que algunas combinaciones importantes pueden no ser evaluadas.

© Universidad Internacional de La Rioja (UNIR)

Optimización bayesiana

Es un enfoque más avanzado que utiliza **modelos probabilísticos** para buscar los mejores hiperparámetros. A diferencia de las técnicas anteriores, este método **ajusta los hiperparámetros** de manera inteligente con basa en el rendimiento de las pruebas anteriores.

- La ventaja es que es más eficiente, ya que explora inteligentemente el espacio de hiperparámetros.
- La desventaja es que es más complejo, por lo que puede requerir más recursos computacionales.

Por ejemplo, en la optimización de CNN, la optimización bayesiana demostró ser capaz de encontrar configuraciones óptimas de hiperparámetros en menos iteraciones que la búsqueda en cuadrícula o aleatoria, lo que reduce significativamente el tiempo de entrenamiento.

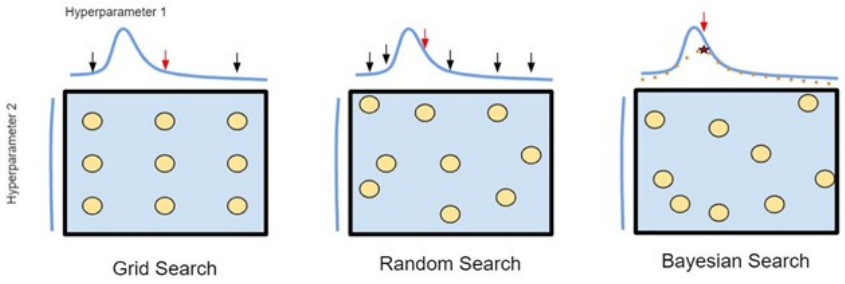


Figura 7. Comparativa de métodos de búsqueda de hiperparámetros. Fuente: Dua y Patel, 2024.

© Universidad Internacional de La Rioja (UNIR)

Mejora continua de modelos

Una vez que se entrenó un modelo con éxito, no se debe considerar como un proceso terminado. Los entornos de datos cambian, lo que puede degradar el rendimiento del modelo si no se actualiza o ajusta de manera continua.

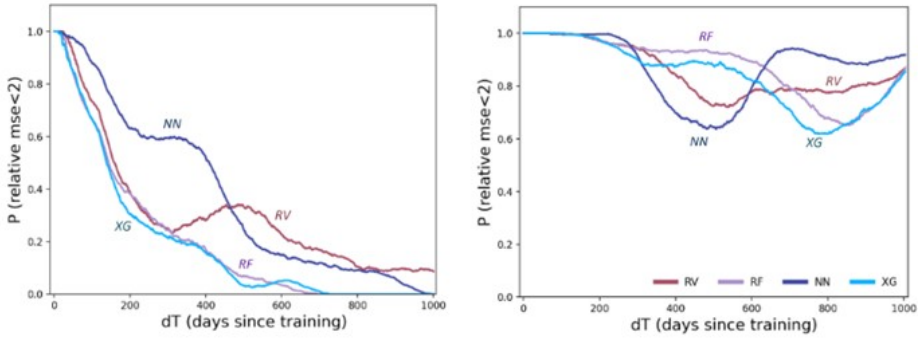


Figura 8. Variación del rendimiento en modelos aplicados en medicina a lo largo del tiempo. Fuente: Vela et al., 2022.

Retraining **(reentrenamiento)**

A medida que se recopilan más datos, es fundamental **reentrenar el modelo** para que mantenga su precisión. Esto es especialmente relevante en campos como el comercio electrónico o los sistemas de recomendación, donde los **patrones de los usuarios cambian constantemente.**

© Universidad Internacional de La Rioja (UNIR)

Monitorización del rendimiento

Los modelos deben ser monitoreados constantemente para asegurarse de que **continúan funcionando bien.** Si el rendimiento disminuye, puede ser necesario ajustar los hiperparámetros o incluso rediseñar el modelo.

Ajuste dinámico de hiperparámetros

En algunos sistemas, como los que **trabajan en tiempo real** o con **grandes volúmenes de datos** en constante cambio, los hiperparámetros pueden **ajustarse dinámicamente** a medida que el modelo se utiliza. Por ejemplo, los modelos de aprendizaje en línea ajustan los hiperparámetros de forma continua a medida que se introducen nuevos datos.

Por ejemplo, en los sistemas de predicción financiera, el rendimiento de los modelos debe ajustarse continuamente, dado que las condiciones del mercado cambian con frecuencia. Los modelos que se reentrenan periódicamente con nuevos datos tienen más posibilidades de proporcionar predicciones precisas.

Caso práctico: optimización de un algoritmo de aprendizaje supervisado mediante GA e IA

- Objetivo. Optimizar los hiperparámetros de un modelo de clasificación SVM a través de un GA. El objetivo es maximizar la precisión del modelo seleccionando de forma automática los mejores hiperparámetros.
- Escenario. Una empresa de análisis financiero desea predecir si una transacción es fraudulenta o no. Se usa el conjunto de datos Breast Cancer de Scikit-Learn para demostrar el proceso. Los hiperparámetros a optimizar son:
  - C : parámetro de regularización (rango: [0.01, 100]).
  - gamma : coeficiente de la función de *kernel* (rango: [0.0001, 1]).
  - kernel : tipo de *kernel* (lineal, polinomial o radial).

© Universidad Internacional de La Rioja (UNIR)

Para encontrar la mejor combinación de hiperparámetros, se utilizará un GA que seleccionará y evolucionará las mejores combinaciones posibles.

Instalación de bibliotecas

Asegúrate de tener Scikit-Learn y DEAP *(distributed evolutionary algorithms in Python)*.

```
$> pip install scikit-learn deap
```

Preparación de los datos

Importación de librerías y carga de datos:

```
import numpy as np

import pandas as pd

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score



# Cargar el conjunto de datos Breast Cancer

data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)

y = data.target
```


© Universidad Internacional de La Rioja (UNIR)

```
# Dividir los datos en conjuntos de entrenamiento (80%) y prueba (20%)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)



print(f"Tamaño de los datos de entrenamiento: {X_train.shape}")

print(f"Tamaño de los datos de prueba: {X_test.shape}")

4. Entrenamiento Inicial del Modelo (Baseline)

Se entrena un SVM con parámetros predeterminados para tener una referencia
inicial de rendimiento.

from sklearn.svm import SVC

# Entrenar modelo SVM con parámetros predeterminados

clf_baseline = SVC(random_state=42)

clf_baseline.fit(X_train, y_train)



# Evaluar precisión en el conjunto de prueba

y_pred = clf_baseline.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)



print(f"Precisión del modelo base (sin optimización): {accuracy:.4f}")

Resultado esperado:

Precisión del modelo base (sin optimización): 0.9211
```


© Universidad Internacional de La Rioja (UNIR)

Implementación del algoritmo genético para la optimización

- Hiperparámetros que hay que optimizar:
  - C : parámetro de regularización (rango: [0.01, 100])
  - gamma : parámetro de la función de *kernel* (rango: [0.0001, 1])
  - kernel : tipo de kernel (valores: ['linear', 'poly', 'rbf'])
- Definición del GA:
  - Población. Cada individuo representa una combinación de hiperparámetros.
  - Selección. Selección por torneo (los mejores pasan a la siguiente generación).
  - Cruce. Se cruzan los valores de los hiperparámetros.
  - Mutación. Se aplican pequeñas modificaciones en los hiperparámetros de los individuos.

Código del algoritmo genético

```
import random

from deap import base, creator, tools

# Definir la función de evaluación (fitness)

def evaluar_hyperparametros(individuo):

    C, gamma, kernel_index = individuo

    kernel_options = ['linear', 'poly', 'rbf']

    kernel = kernel_options[int(kernel_index)]
```


© Universidad Internacional de La Rioja (UNIR)

```
    # Crear modelo SVM con los hiperparámetros actuales
    clf = SVC(C=C, gamma=gamma, kernel=kernel, random_state=42)
    clf.fit(X_train, y_train)

    # Evaluar la precisión en el conjunto de prueba
    y_pred = clf.predict(X_test)
    precision = accuracy_score(y_test, y_pred)
    return precision, # La coma indica que es una tupla
# Crear la clase Fitness y los Individuos
creator.create("FitnessMax", base.Fitness, weights=(1.0,)) # Maximizar la
precisión
creator.create("Individual", list, fitness=creator.FitnessMax)
# Definir la población
toolbox = base.Toolbox()
toolbox.register("attr_C", random.uniform, 0.01, 100) # Rango de C
toolbox.register("attr_gamma", random.uniform, 0.0001, 1) # Rango de gamma
toolbox.register("attr_kernel", random.randint, 0, 2) # kernel (0: linear,
1: poly, 2: rbf)
toolbox.register("individual", tools.initCycle, creator.Individual,
                 (toolbox.attr_C, toolbox.attr_gamma, toolbox.attr_kernel),
n=1)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
```


© Universidad Internacional de La Rioja (UNIR)

```
# Definir las operaciones genéticas

toolbox.register("mate", tools.cxBlend, alpha=0.5) # Cruce

toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=0.1, indpb=0.2) #
Mutación

toolbox.register("select", tools.selTournament, tournsize=3) # Selección

toolbox.register("evaluate", evaluar_hyperparametros)
```

Ejecución del algoritmo genético

```
# Configuración de la población

population = toolbox.population(n=10) # Población inicial de 10 individuos

NGEN = 20 # Número de generaciones

CXPB, MUTPB = 0.5, 0.2 # Probabilidad de cruce y mutación

# Evolucionar la población

for gen in range(NGEN):

    print(f"Generación {gen}")



    # Evaluar la población

    fitnesses = list(map(toolbox.evaluate, population))

    for ind, fit in zip(population, fitnesses):

        ind.fitness.values = fit

    # Selección

    offspring = toolbox.select(population, len(population))
```


© Universidad Internacional de La Rioja (UNIR)

```
    offspring = list(map(toolbox.clone, offspring))


    # Cruce y mutación
    for child1, child2 in zip(offspring[::2], offspring[1::2]):
        if random.random() < CXPB:
            toolbox.mate(child1, child2)
            del child1.fitness.values
            del child2.fitness.values
    for mutant in offspring:
        if random.random() < MUTPB:
            toolbox.mutate(mutant)
            del mutant.fitness.values
    population[:] = offspring
# Mostrar los mejores hiperparámetros
best_individual = tools.selBest(population, 1)[0]
print(f"Mejores Hiperparámetros: C={best_individual[0]}, gamma=
{best_individual[1]}, kernel={['linear', 'poly', 'rbf']
[int(best_individual[2])]}")
```


© Universidad Internacional de La Rioja (UNIR)

Resultado posible

```
Mejores Hiperparámetros: C=15.23, gamma=0.04, kernel=rbf

Mejor Precisión en Pruebas: 0.9512
```

Modelo	Precisión en pruebas
SVM (base)	0.9211
GA optimizado	0.9512

Tabla 2. Resultado posible. Fuente: elaboración propia.

Conclusiones

- Mejora de precisión:
  - El modelo SVM inicial obtuvo una precisión de 92,11 %.
  - La optimización con GA alcanzó una precisión de 95,12 %.
- Ajuste eficiente de hiperparámetros. Los GA permiten buscar soluciones óptimas en menos tiempo que *grid search.*
- Impacto. El uso de GA mejora la eficiencia y evita la exploración completa del espacio de soluciones.

© Universidad Internacional de La Rioja (UNIR)

6.6. Evaluación del rendimiento con optimización de IA

La evaluación del rendimiento de algoritmos optimizados por IA es crucial para determinar la **eficiencia, escalabilidad y adecuación** de los modelos implementados. En el contexto de desarrollo de *software,* la IA se utiliza para **mejorar el rendimiento** en múltiples aspectos: desde la velocidad de procesamiento y la eficiencia del uso de recursos hasta la **precisión de modelos predictivos o generativos.**

La optimización de algoritmos mediante IA es un **proceso complejo** que ofrece mejoras significativas en múltiples aspectos del rendimiento de *software,* desde la reducción de tiempos de ejecución hasta la mejora en el uso de recursos. Evaluar correctamente el impacto de las técnicas de optimización de IA sirve para garantizar que estas optimizaciones realmente **aporten valor** y que las técnicas como el *benchmarking,* las pruebas de estrés y la monitorización en tiempo real permitan identificar las áreas donde la IA puede **generar los mayores beneficios.**

Métricas clave en la evaluación del rendimiento

Para evaluar el rendimiento de los algoritmos optimizados con IA, se deben tener en cuenta diversas **métricas clave.** Estas métricas permiten analizar desde diferentes perspectivas **cómo afecta la optimización a la eficiencia general** del sistema.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Tiempo de ejecución *(execution time).*** El tiempo de ejecución mide la cantidad de tiempo que tarda el algoritmo en completarse. En algoritmos optimizados por IA, una reducción en el tiempo de ejecución es un indicativo clave de mejora en el rendimiento.

  - Por ejemplo, en un sistema de procesamiento de grandes volúmenes de datos, el uso de un modelo de IA que optimiza el procesamiento paralelo puede reducir el tiempo de ejecución de varias horas a solo minutos.

- **Consumo de recursos.** El uso de CPU *(central processing unit)*, de memoria y de energía son métricas fundamentales cuando se evalúa la eficiencia de los algoritmos optimizados. Los algoritmos de optimización de IA suelen buscar reducir el uso de estos recursos, especialmente en entornos donde los recursos son limitados, como en sistemas embebidos o aplicaciones de tiempo real.

  - Por ejemplo, en aplicaciones de Internet de las cosas (IoT), la IA puede optimizar el código para reducir el consumo de energía, lo que resulta en una mayor duración de la batería en dispositivos remotos.

- **Escalabilidad.** La escalabilidad mide cómo el rendimiento de un algoritmo cambia a medida que se incrementa el tamaño de los datos o el número de usuarios. Hay algoritmos optimizados con IA que deben ser capaces de manejar de manera eficiente mayores volúmenes de información sin una disminución significativa en el rendimiento (tiempo de ejecución, consumo de recursos, etc.).

  - Por ejemplo, un sistema de gestión de bases de datos optimizado con IA puede distribuir consultas de manera más eficiente a medida que aumenta el número de usuarios concurrentes.

© Universidad Internacional de La Rioja (UNIR)

- **Precisión y exactitud.** Para algoritmos de *machine learning* o de optimización predictiva, la precisión es fundamental. Evaluar el impacto de la optimización implica medir si los cambios introducidos afectan a la capacidad del modelo para predecir correctamente o ejecutar una tarea específica.

  - Por ejemplo, en un modelo de redes neuronales optimizado para reconocimiento de imágenes, la precisión se evalúa al comparar los resultados antes y después de la optimización, lo que asegura que el rendimiento no se degrade.

Pruebas de rendimiento y validación

Una vez aplicadas las técnicas de optimización, es necesario realizar pruebas para validar los beneficios obtenidos.

Algunas de las pruebas más comunes son:

- El *benchmarking* compara el rendimiento de un algoritmo optimizado con un conjunto estándar de algoritmos no optimizados. Estas pruebas permiten cuantificar los beneficios de la optimización.

  - Por ejemplo, un sistema de bases de datos distribuido optimizado con IA puede ser evaluado en términos de tiempos de consulta y manejo de datos en comparación con una versión no optimizada.

- Las pruebas de estrés evalúan cómo se comporta un algoritmo optimizado bajo condiciones extremas, como altos volúmenes de datos o picos de usuarios. Esto asegura que la optimización sea eficiente incluso en escenarios fuera de lo común.

  - Por ejemplo, en un servidor de aplicaciones web optimizado por IA, las pruebas de estrés simulan millones de peticiones simultáneas para evaluar si la optimización mejora el rendimiento bajo carga pesada.

© Universidad Internacional de La Rioja (UNIR)

Monitorización del rendimiento

El ciclo de mejora continua implica **monitorear y ajustar constantemente** los algoritmos optimizados. A medida que las condiciones de uso cambian o se introducen nuevos datos, es importante **reevaluar el rendimiento** del sistema.

Los sistemas optimizados deben ser monitoreados en tiempo real para **detectar cualquier caída** en el rendimiento o anomalías que puedan surgir. Las técnicas de IA pueden incluso **ajustar los parámetros dinámicamente,** mientras que el sistema está en ejecución.

Por ejemplo, en los sistemas de recomendación, como los utilizados por plataformas de *streaming,* la IA ajusta el sistema en tiempo real según el comportamiento de los usuarios, por lo que mejora la precisión de las recomendaciones y el rendimiento del servidor.

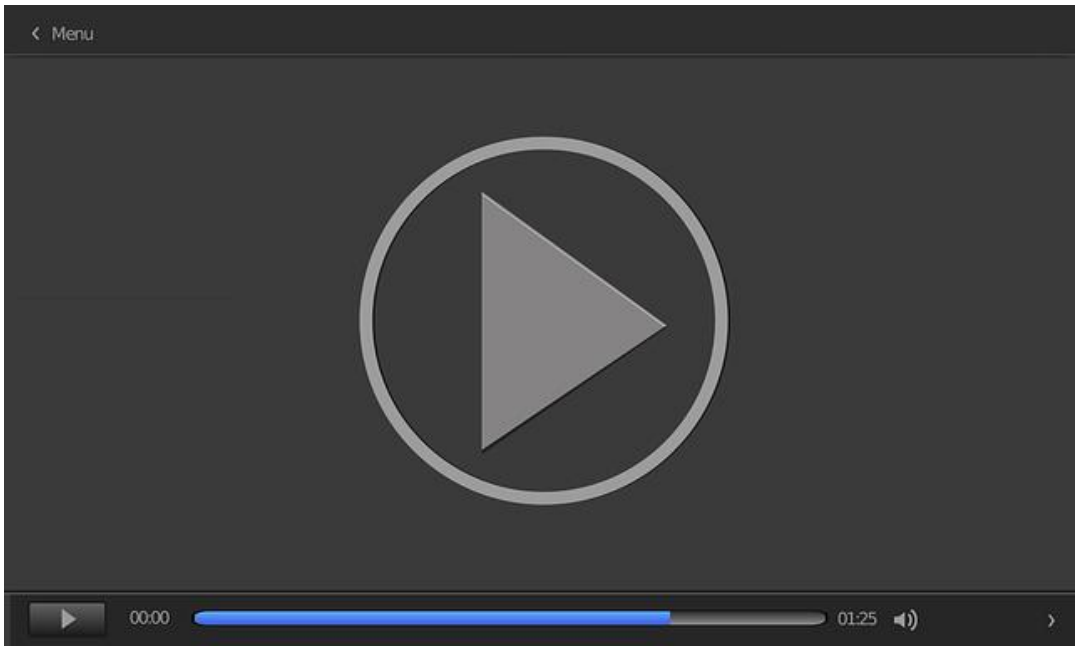
- Un *fine-tune adapter* es un conjunto de parámetros adicionales o una pequeña red que se añade al modelo original que se entrena durante el proceso de ajuste fino *(fine-tuning)*. En lugar de modificar todos los pesos y los sesgos del modelo base, se congela la mayor parte del modelo y solo se entrenan estos parámetros adicionales, lo que reduce significativamente la cantidad de cálculos y recursos necesarios.

- En un escenario en el que se requiera adaptar el comportamiento de un LLM rápidamente para diferentes tareas o dominios, los *adapters* permiten hacer cambios rápidos sin necesidad de volver a entrenar el modelo completo.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

A continuación, veremos el vídeo *Algoritmos evolutivos en desarrollo Software.* Este vídeo introduce los algoritmos evolutivos y su aplicación en la optimización de *software,* un área avanzada y emergente. Se explicará cómo funcionan los algoritmos, como los algoritmos genéticos y la optimización por enjambre, para mejorar el rendimiento del código. Asimismo, se mostrarán ejemplos de uso en problemas complejos de *software,* como la optimización de parámetros en grandes sistemas, y su impacto en el ciclo de desarrollo.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=eb904f78-de42-47d3-9534-b242012aaf01

© Universidad Internacional de La Rioja (UNIR)

6.7. Referencias bibliográficas

Cummins, C., Seeker, V., Grubisic, D., Roziere, B., Gehring, J., Synnaeve, G. y Leather, H. (2024). Meta large language model compiler: Foundation models of compiler optimization. *ArXiv.*

Dua, I. K. y Patel, P. G. (2024). Model Training and Inference Optimization. En *Optimizing Generative AI Workloads for Sustainability.* APress.

Fouad, Z., Alfonse, M., Roushdy, M. y Salem, A. B. M. (2021). Hyper-parameter optimization of convolutional neural network based on particle swarm optimization algorithm. *Bulletin of Electrical Engineering and Informatics, 10*(6), pp. 3377-3384.

Li, K. (2024, septiembre 23). How to Choose a Learning Rate Scheduler for Neural Networks. *Neptune.ai.* https://neptune.ai/blog/how-to-choose-a-learning-rate-scheduler

Nutakki, M. y Mandava, S. (2023). Review on optimization techniques and role of Artificial Intelligence in home energy management systems. *Engineering Applications of Artificial Intelligence, 119.*

Ozdemir, A., Lim, J. Y. y Singh, C. (2005). Post-outage reactive power flow calculations by genetic algorithms: constrained optimization approach. *IEEE Transactions on Power Systems, 20*(3), pp. 1266-1272.

Vela, D., Sharp, A., Zhang, R., Nguyen, T., Hoang, A. y Pianykh, O. S. (2022). Temporal quality degradation in AI models. *Scientific Reports, 12*(1).

Yeom, S. K., Seegerer, P., Lapuschkin, S., Binder, A., Wiedemann, S., Müller, K. R. y Samek, W. (2021). Pruning by explaining: A novel criterion for deep neural network pruning. *Pattern Recognition, 115.*

© Universidad Internacional de La Rioja (UNIR)

Algoritmos para problemas de optimización

Tang, J., Liu, G. y Pan, Q. (2021). A review on representative swarm intelligence algorithms for solving optimization problems: Applications and trends. *IEEE/CAA Journal of Automatica Sinica, 8*(10), pp. 1627-1643. https://ieeexplore.ieee.org/iel7/6570654/9497871/09498989.pdf

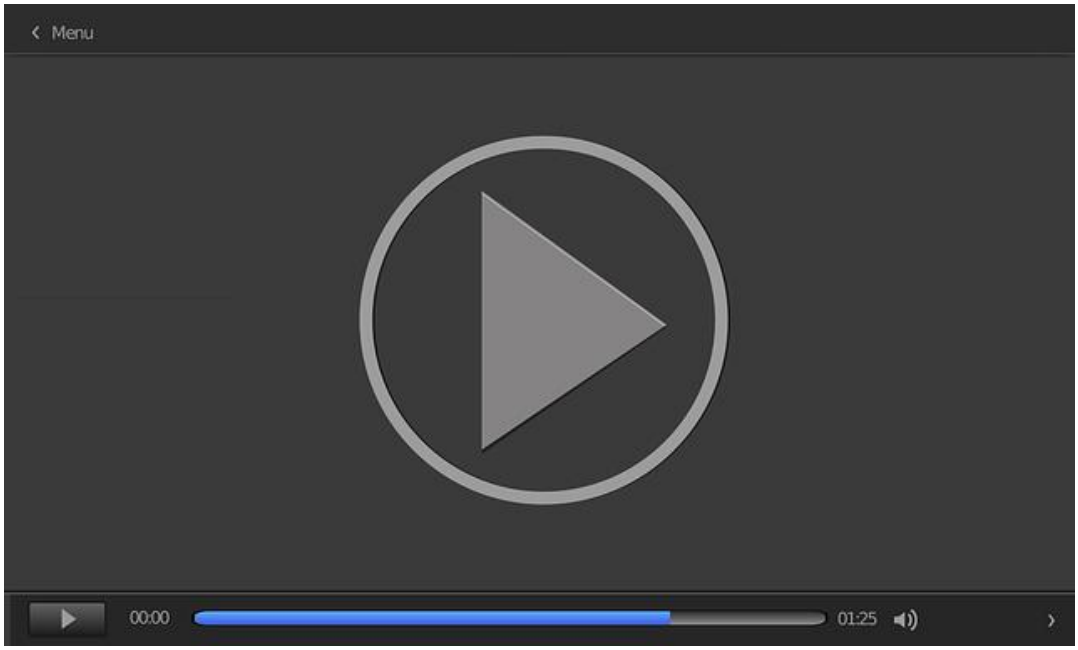
Este artículo revisa los últimos avances en la optimización de algoritmos mediante técnicas de IA, como algoritmos evolutivos y por enjambre.

© Universidad Internacional de La Rioja (UNIR)

Algoritmos genéticos con ejemplos

Kie Codex. (2020, julio 13). Genetic Algorithms Explained By Example [Vídeo]. YouTube. https://www.youtube.com/watch?v=uQj5UNhCPuo

Un vídeo introductorio que explica cómo funcionan los algoritmos genéticos y muestra ejemplos prácticos de cómo se utilizan para resolver problemas de optimización en diferentes dominios.



Accede al vídeo:

https://www.youtube.com/embed/uQj5UNhCPuo?si=28wjvTpK2DHwnaDe

© Universidad Internacional de La Rioja (UNIR)

Optimización de código con IA

Codecademy Team. (s. f.). How to Optimize Code with Generative AI. *Codecademy*. https://www.codecademy.com/article/how-to-optimize-code-with-generative-ai

En esta guía de Codacademy, se detalla cómo aplicar IA generativa para optimizar código en Python mediante ejemplos.

© Universidad Internacional de La Rioja (UNIR)

Ajuste de hiperparámetros automática con Amazon SageMaker

Yoldas, Ü, Lesyk, E. y Kapm, M. (2023, noviembre 10). Explore advanced techniques for hyperparameter optimization with Amazon SageMaker Automatic Model Tuning. *AWS Machine Learning Blog.* https://aws.amazon.com/es/blogs/machine-learning/explore-advanced-techniques-for-hyperparameter-optimization-with-amazon-sagemaker-automatic-model-tuning/

Artículo en el blog oficial de Amazon AWS sobre los avances en la optimización de hiperparámetros y el uso de algoritmos de búsqueda eficientes en el ajuste de modelos de aprendizaje automático.

© Universidad Internacional de La Rioja (UNIR)

Test

1. ¿Cuál es uno de los beneficios clave de la optimización de algoritmos con IA?

A. Aumenta el consumo de recursos.

B. Mejora la eficiencia y el rendimiento.

C. Solo se aplica en *hardware* especializado.

D. Solo optimiza el diseño gráfico.

2. ¿Qué técnica de optimización está inspirada en el comportamiento de animales como bandadas de pájaros?

A. El recocido simulado.

B. Los GA.

C. La PSO.

D. El aprendizaje supervisado.

3. ¿Qué método ajusta automáticamente los parámetros externos de un modelo de IA?

A. El entrenamiento supervisado.

B. El ajuste de hiperparámetros.

C. El aprendizaje no supervisado.

D. La inferencia automática.

4. ¿Cuál es una característica principal de los GA?

A. Están basados en la teoría de la relatividad.

B. Utilizan una población inicial de soluciones.

C. Solo se aplican en sistemas de bases de datos.

D. Optimizan solo un parámetro a la vez.

© Universidad Internacional de La Rioja (UNIR)

5. ¿Qué técnica busca soluciones cercanas al óptimo en problemas complejos?

A. La búsqueda exhaustiva.

B. Los algoritmos meméticos.

C. Las pruebas manuales.

D. La optimización basada en heurísticas.

6. ¿Qué tipo de IA aprende a optimizar algoritmos mediante recompensas o penalizaciones?

A. El aprendizaje supervisado.

B. El aprendizaje por refuerzo.

C. La optimización lineal.

D. El aprendizaje basado en modelos.

7. ¿Qué técnica se usa para reducir el número de parámetros en redes neuronales?

A. La expansión de redes.

B. La poda *(pruning)*.

C. La agregación de datos.

D. La cuantización.

8. ¿Qué estrategia permite ajustar hiperparámetros de manera más eficiente que la búsqueda en cuadrícula?

A. La búsqueda aleatoria.

B. Los GA.

C. La optimización bayesiana.

D. El recocido simulado.

© Universidad Internacional de La Rioja (UNIR)

9. ¿Qué técnica optimiza el consumo de energía en sistemas de *software*?

A. La PSO.

B. El ajuste de parámetros manual.

C. La cuantización.

D. El uso de análisis heurístico.

10. ¿Qué método de optimización está inspirado en el proceso evolutivo de Darwin?

A. El recocido simulado.

B. La optimización bayesiana.

C. Los GA.

D. La búsqueda tabú.

11. ¿Cuál es el objetivo principal del ajuste de hiperparámetros en modelos de IA?

A. Disminuir el tamaño del conjunto de datos.

B. Aumentar el número de parámetros.

C. Maximizar el rendimiento del modelo.

D. Reducir la complejidad del código.

12. ¿Qué métrica se utiliza para evaluar la escalabilidad de un algoritmo optimizado?

A. La precisión.

B. El tiempo de respuesta.

C. La tasa de aprendizaje.

D. El uso de CPU.

© Universidad Internacional de La Rioja (UNIR)

13. ¿Qué técnica de optimización ajusta la precisión de los números en redes neuronales para mejorar la velocidad?

A. El recocido simulado.

B. La poda.

C. La cuantización.

D. La expansión de modelos.

14. ¿Qué técnica es común en la optimización de tareas de planificación en sistemas de nube?

A. El ajuste de hiperparámetros manual.

B. Los GA.

C. La búsqueda en cuadrícula.

D. La optimización lineal.

15. ¿Qué estrategia se usa para optimizar CNN al eliminar las conexiones innecesarias?

A. Expansión de capas.

B. Recocido simulado.

C. El *pruning.*

D. Agregación de nodos.

16. ¿Cuál es una desventaja de la búsqueda en cuadrícula para el ajuste de hiperparámetros?

A. No evalúa todas las combinaciones posibles.

B. Consume mucho tiempo y recursos.

C. Requiere menos recursos computacionales.

D. Solo funciona en modelos supervisados.

© Universidad Internacional de La Rioja (UNIR)

17. ¿Qué técnica permite ajustar los parámetros de un modelo en tiempo real?

A. La búsqueda tabú.

B. El aprendizaje supervisado.

C. El ajuste dinámico de hiperparámetros.

D. La expansión de datos.

18. ¿Qué método se usa para medir la eficiencia del uso de recursos en algoritmos optimizados?

A. Las pruebas manuales.

B. El tiempo de ejecución.

C. El análisis de costos.

D. El consumo de memoria.

19. ¿Qué técnica de optimización permite reducir la complejidad de un modelo sin comprometer significativamente su precisión?

A. La agregación de datos.

B. La cuantización.

C. La ampliación de redes.

D. La búsqueda exhaustiva.

20. ¿Qué técnica de aprendizaje se utiliza para ajustar políticas de decisión secuenciales en tiempo real?

A. La búsqueda en cuadrícula.

B. El aprendizaje por refuerzo.

C. La expansión de modelos.

D. La poda.

© Universidad Internacional de La Rioja (UNIR)